

MAMBA-Frost x86 Implementation Self-Evaluation Report

MAMBA-Frost Submission Team

June 28, 2026

Document Information

Algorithm name	MAMBA-Frost
Algorithm category	Public-key cryptography: Key Encapsulation Mechanism (KEM)
Target architecture	x86-64
Implementation versions	Reference implementation; performance-optimized implementation
Submission directory	API_PKC/
Code version	Branch codex/-api_pkc, commit 37fadadba296f300caed150192c8bcd86d5ba4c0
Report status	x86 self-evaluation report for run 20260628-150128

1 Scope of This Report

This report gives the x86 implementation self-evaluation of MAMBA-Frost. The submitted implementation package is organized as a single `API_PKC/` package. It contains the default MAMBA-Frost instances and the compact-ciphertext MAMBA-Frost-CC instances. The latter are implementation instances of the same MAMBA-Frost algorithm family and are reported as a compact-ciphertext variant rather than as a separate algorithm. Each algorithm instance is placed in an independent directory under the Reference and Optimized implementation trees.

This report covers two implementation versions.

- **Reference implementation.** The reference implementation is located under `API_PKC/Implementations/Reference/`. It is written in ISO C99 and does not rely on AVX2, AES-NI, inline assembly, or other platform-specific instruction sets.
- **Performance-optimized implementation.** The optimized implementation is located under `API_PKC/Implementations/Optimized_Implementation/`. It targets x86-64 processors with AVX2 and AES-NI support.

The evaluation covers the following ten KEM instances of MAMBA-Frost.

Instance	Target	Public key	Secret key	Shared secret	Ciphertext
MAMBA-Frost-128	128	5152	6736	16	5192
MAMBA-Frost-192	192	9712	11528	24	9760
MAMBA-Frost-256	256	16776	19416	32	15552
MAMBA-Frost-384	384	25096	29032	48	37736
MAMBA-Frost-512	512	36432	41728	64	72944
MAMBA-Frost-CC-128	128	5152	6736	16	5192
MAMBA-Frost-CC-192	192	9712	11528	24	9760
MAMBA-Frost-CC-256	256	16776	19416	32	15552
MAMBA-Frost-CC-384	384	37628	43492	48	25204
MAMBA-Frost-CC-512	512	72832	83328	64	36544

All sizes are given in bytes. The compact-ciphertext instances at security levels 128, 192, and 256 use the same external byte lengths as the corresponding default instances. The compact-ciphertext instances at security levels 384 and 512 use the compact-ciphertext layout.

2 Implementation Materials

2.1 Submission Directory Layout

The formal implementation submission boundary is:

```
API_PKC/
```

This directory is a self-contained API_PKC submission package. It contains the submitted implementations, formal KAT outputs, internal correctness tests, dependency documentation, and self-evaluation support files.

The main directory layout is as follows:

```
API_PKC/
  README.md
  DEPENDENCIES.txt
  SELF_EVALUATION.md
  Makefile
  Implementations/
    README.md
    Reference_Implementation/
      MAMBA-Frost-128/
      MAMBA-Frost-192/
      MAMBA-Frost-256/
      MAMBA-Frost-384/
      MAMBA-Frost-512/
      MAMBA-Frost-CC-128/
      MAMBA-Frost-CC-192/
      MAMBA-Frost-CC-256/
      MAMBA-Frost-CC-384/
      MAMBA-Frost-CC-512/
    Optimized_Implementation/
      MAMBA-Frost-128/
      MAMBA-Frost-192/
      MAMBA-Frost-256/
      MAMBA-Frost-384/
      MAMBA-Frost-512/
      MAMBA-Frost-CC-128/
      MAMBA-Frost-CC-192/
      MAMBA-Frost-CC-256/
      MAMBA-Frost-CC-384/
```

```
MAMBA-Frost-CC-512/  
Additional_Implementation/  
  README.md  
Test_Vector/  
  KAT_KEM_MAMBA-Frost-128.txt  
  KAT_KEM_MAMBA-Frost-192.txt  
  KAT_KEM_MAMBA-Frost-256.txt  
  KAT_KEM_MAMBA-Frost-384.txt  
  KAT_KEM_MAMBA-Frost-512.txt  
  KAT_KEM_MAMBA-Frost-CC-128.txt  
  KAT_KEM_MAMBA-Frost-CC-192.txt  
  KAT_KEM_MAMBA-Frost-CC-256.txt  
  KAT_KEM_MAMBA-Frost-CC-384.txt  
  KAT_KEM_MAMBA-Frost-CC-512.txt
```

The directory `Additional_Implementation/` is retained only as a placeholder for future implementation variants. It is not part of the reference or performance-optimized x86 test results reported here.

2.2 Use of Official API_PKC Support Files

Each `MAMBA-Frost-*` and `MAMBA-Frost-CC-*` instance directory provides its own:

- `KEM_AlgorithmInstance.h`
- `KEM_AlgorithmInstance.c`
- `KAT_KEM.c`
- `drng.c`
- `drng.h`
- `auxfunc.c`
- `auxfunc.h`
- `Makefile`
- local API compatibility and core source files required to build the instance

The `KEM_AlgorithmInstance.h` and `KEM_AlgorithmInstance.c` files implement the official API_PKC KEM interface. The official KAT generator `KAT_KEM.c`, official DRNG files, and official auxiliary-function files are included in each instance directory.

The following command checks the presence of the API_PKC support files in all instance directories:

```
find API_PKC/Implementations -path "*MAMBA-Frost*" -maxdepth 4 -type f \  
  \( -name "KEM_AlgorithmInstance.h" -o -name "KEM_AlgorithmInstance.c" \  
    -o -name "KAT_KEM.c" -o -name "drng.c" -o -name "drng.h" \  
    -o -name "auxfunc.c" -o -name "auxfunc.h" \) | sort
```

Result: **PASS**.

2.3 Randomness, Hash, and XOF Integration

In the `API_PKC` package, the byte-oriented `randombytes` interface used by the KEM core is adapted to the official DRNG. The compatibility symbol `randombytes` eventually calls:

```
get_random_number(&drng_algorithm, x, xlen * 8ULL)
```

The `API_PKC` packaging path does not use `rand()`, `/dev/urandom`, `getrandom()`, `arc4random()`, OpenSSL RNG, or the development test RNG.

The hash and XOF compatibility layer routes the SHAKE-style call sites used by the Frost core to the official `API_PKC` auxiliary function `pseudoXOF`. The compatibility mapping is:

- `shake128` is routed to `pseudoXOF`.
- `shake256` is routed to `pseudoXOF`.

The default Reference and Optimized build paths do not use a standalone FIPS 202 SHAKE implementation as the default XOF provider. The optimized AES-128 matrix-expansion path does not compile the complete Keccak times4 implementation by default.

3 x86 Test Environment

3.1 Hardware Environment

CPU model	AMD Ryzen 7 9700X 8-Core Processor
Architecture	x86-64
CPU frequency	Not separately recorded; BogoMIPS reported as 7599.83
CPU count	16 logical CPUs
CPU byte order	Little Endian
Instruction sets	SSE2, AES, PCLMULQDQ, AVX, AVX2, BMI1, BMI2, SHA-NI, AVX-512 extensions
AVX2 availability	Yes
AES-NI availability	Yes
Number of cores used	Single-threaded benchmark execution
Memory	15 GiB RAM; 2.0 GiB swap
Cache information	L1d: 768 KiB; L1i: 512 KiB; L2: 16 MiB; L3: 256 MiB

3.2 Software Environment

Operating system	Ubuntu 22.04.5 LTS
Linux kernel	Linux 6.8.0-124-generic, x86-64
Compiler	GCC 11.4.0
Build tool	GNU Make 4.3
CMake	CMake 3.22.1

Timing method	Serialized x86 cycle counter and <code>CLOCK_MONOTONIC_RAW</code> in the self-evaluation benchmark
Test date	2026-06-28T15:01:28Z
Result directory	<code>x86_Self_Evaluation/results/20260628-150128/</code>

4 Compilation Configuration

4.1 Reference Implementation

The reference implementation uses portable ISO C99 code and does not require AVX2, AES-NI, inline assembly, or other architecture-specific instruction sets. The compilation flags are:

```
-std=c99 -Wpedantic -Wall -Wextra -O2
```

The reference implementation preserves compile-time size and layout checks through a C99-compatible `FROST_STATIC_ASSERT` macro. This macro maps to `_Static_assert` on C11 or later compilers and to a typedef-based compile-time assertion on C99 compilers. It does not change the algorithm semantics, byte layout, or KAT behavior.

Example instance directories:

```
API_PKC/Implementations/Reference_Implementation/MAMBA-Frost-128/
API_PKC/Implementations/Reference_Implementation/MAMBA-Frost-CC-384/
```

The build commands used in the reported run are:

```
cd API_PKC
make clean
make
make smoke
make kats
make kat-repro
make check
```

4.2 Performance-Optimized Implementation

The performance-optimized implementation targets x86-64 processors with AVX2 and AES-NI support. The compilation flags are:

```
-std=c99 -Wpedantic -Wall -Wextra -O3 -march=x86-64 -mavx2 -maes -mtune=native -flto -fomit-frame-pointer
```

The optimized build uses the `API_PKC` KEM interface and exposes the same external byte lengths as the corresponding reference implementation. It does not change the KEM serialization format.

Example instance directories:

```
API_PKC/Implementations/Optimized_Implementation/MAMBA-Frost-128/
API_PKC/Implementations/Optimized_Implementation/MAMBA-Frost-CC-384/
```

The build commands used in the reported run are:

```
cd API_PKC
make optimized-clean
make optimized-all
make optimized-smoke
make optimized-kats
make optimized-kat-repro
make optimized-check
```

5 Functional Correctness Tests

5.1 Known-Answer Tests

The formal KAT files are located under:

```
API_PKC/Test_Vector/
```

The files are:

```
KAT_KEM_MAMBA-Frost-128.txt
KAT_KEM_MAMBA-Frost-192.txt
KAT_KEM_MAMBA-Frost-256.txt
KAT_KEM_MAMBA-Frost-384.txt
KAT_KEM_MAMBA-Frost-512.txt
KAT_KEM_MAMBA-Frost-CC-128.txt
KAT_KEM_MAMBA-Frost-CC-192.txt
KAT_KEM_MAMBA-Frost-CC-256.txt
KAT_KEM_MAMBA-Frost-CC-384.txt
KAT_KEM_MAMBA-Frost-CC-512.txt
```

These files are generated by the official `KAT_KEM.c`. The KAT procedure covers key generation, encapsulation, and decapsulation for all ten instances.

Instance	Reference KAT	Optimized KAT
MAMBA-Frost-128	PASS	PASS
MAMBA-Frost-192	PASS	PASS
MAMBA-Frost-256	PASS	PASS
MAMBA-Frost-384	PASS	PASS
MAMBA-Frost-512	PASS	PASS
MAMBA-Frost-CC-128	PASS	PASS
MAMBA-Frost-CC-192	PASS	PASS
MAMBA-Frost-CC-256	PASS	PASS
MAMBA-Frost-CC-384	PASS	PASS
MAMBA-Frost-CC-512	PASS	PASS

The following commands are used to generate and check the KAT files:

```
cd API_PKC
make kats
make kat-repro
make optimized-kats
make optimized-kat-repro
```

Conclusion: all ten Reference instances and all ten Optimized instances pass KAT generation and byte-for-byte KAT reproducibility.

5.2 API Wrapper Tests

The self-evaluation harness checks the official `API_PKC` KEM wrapper through the smoke tests, full correctness checks, and the API-level benchmark. The checked behavior includes:

- `kem_get_pk_len_bytes`
- `kem_get_sk_len_bytes`
- `kem_get_ss_len_bytes`
- `kem_get_ct_len_bytes`

- output length writeback by `kem_keygen`
- output length writeback by `kem_enc`
- output length writeback by `kem_dec`
- shared-secret consistency for valid encapsulation and decapsulation
- implicit rejection behavior for valid-length invalid ciphertexts

Instance	Reference API check	Optimized API check
MAMBA-Frost-128	PASS	PASS
MAMBA-Frost-192	PASS	PASS
MAMBA-Frost-256	PASS	PASS
MAMBA-Frost-384	PASS	PASS
MAMBA-Frost-512	PASS	PASS
MAMBA-Frost-CC-128	PASS	PASS
MAMBA-Frost-CC-192	PASS	PASS
MAMBA-Frost-CC-256	PASS	PASS
MAMBA-Frost-CC-384	PASS	PASS
MAMBA-Frost-CC-512	PASS	PASS

5.3 Smoke and Full Correctness Tests

Two correctness-test modes are supported by the submission package.

Smoke test	A short test over all ten instances, intended for fast regression checking.
Full check	A longer test over all ten instances, intended for final extended correctness evidence.

The commands are:

```
cd API_PKC
make smoke
make optimized-smoke
make check
make optimized-check
```

Test	Reference	Optimized
Smoke correctness test	PASS	PASS
Full correctness test	PASS	PASS

In the reported run, the full checks were enabled. The full Reference and Optimized checks executed 1000 keypair/encapsulation/decapsulation rounds per instance and passed for all ten instances.

5.4 Implicit Rejection Test

MAMBA-Frost uses implicit rejection. For a valid-length ciphertext that fails the internal re-encryption check, `kem_dec` does not expose ciphertext validity through its return value. Instead, it outputs a fallback shared secret with the expected shared-secret length.

The test procedure is:

```

keygen
enc
copy ct to ct_bad
flip one byte of ct_bad
call kem_dec(sk, sk_len, ct_bad, ct_len, ss_bad, &ss_bad_len)

```

The expected behavior is:

- `kem_dec` returns through the externally successful implicit-rejection path.
- `ss_bad_len` equals the shared-secret length of the corresponding instance.
- Ciphertext validity is not exposed through a branch-dependent external error path.

Conclusion: **PASS**. The benchmark notes for all decapsulation rows record `valid_length_invalid_ct_return`.

6 Performance Test Methodology

6.1 Tested Operations

The following KEM operations are evaluated:

- `kem_keygen`: key generation.
- `kem_enc`: encapsulation.
- `kem_dec`: decapsulation.

The benchmark uses the final `API_PKC` wrapper functions rather than the internal development API. The measured functions are `kem_keygen`, `kem_enc`, and `kem_dec`.

6.2 Number of Runs and Statistics

Each operation is measured independently for each algorithm instance and each implementation version.

Warm-up runs	100
Measurement runs	100
Reported statistic	Median cycle count
Additional raw statistics	Mean, minimum, maximum, standard deviation, and throughput are recorded in the generated CSV files.
Timing unit	CPU cycles
Throughput unit	ops/s

The benchmark records CPU cycle counts using a serialized x86 cycle counter and also records wall-clock timing using `CLOCK_MONOTONIC_RAW`. The benchmark verifies shared-secret consistency during encapsulation and decapsulation.

7 Performance Results

The tables below report median cycle counts over 100 measured runs after 100 warm-up runs.

7.1 Reference Implementation: Median Cycles

Instance	KeyGen	Encaps	Decaps
MAMBA-Frost-128	37,269,977	40,203,893	39,020,599
MAMBA-Frost-192	106,904,615	124,595,134	110,709,473
MAMBA-Frost-256	229,887,064	234,669,261	235,625,700
MAMBA-Frost-384	515,014,676	560,492,856	562,001,044
MAMBA-Frost-512	932,776,910	1,111,745,831	1,112,164,202
MAMBA-Frost-CC-128	36,088,203	39,578,427	44,560,879
MAMBA-Frost-CC-192	106,905,375	107,737,178	110,467,799
MAMBA-Frost-CC-256	228,306,298	234,792,378	235,609,360
MAMBA-Frost-CC-384	528,168,093	528,999,895	528,756,320
MAMBA-Frost-CC-512	985,683,548	976,593,765	989,676,503

7.2 Performance-Optimized Implementation: Median Cycles

Instance	KeyGen	Encaps	Decaps
MAMBA-Frost-128	966,319	1,560,246	2,103,635
MAMBA-Frost-192	2,262,091	2,909,597	3,873,637
MAMBA-Frost-256	4,045,773	5,613,239	7,390,080
MAMBA-Frost-384	7,887,110	13,554,688	16,465,045
MAMBA-Frost-512	13,740,124	27,943,837	34,348,219
MAMBA-Frost-CC-128	927,940	1,564,807	2,104,775
MAMBA-Frost-CC-192	2,228,652	2,830,940	3,597,763
MAMBA-Frost-CC-256	4,049,193	5,493,542	7,216,805
MAMBA-Frost-CC-384	10,904,625	11,365,555	14,634,245
MAMBA-Frost-CC-512	23,841,446	21,446,738	28,721,300

7.3 Optimized-over-Reference Speedup

The following table reports the speedup of the optimized implementation over the reference implementation, computed from the median cycle counts.

Instance	KeyGen	Encaps	Decaps
MAMBA-Frost-128	38.57x	25.77x	18.55x
MAMBA-Frost-192	47.26x	42.82x	28.58x
MAMBA-Frost-256	56.82x	41.81x	31.88x
MAMBA-Frost-384	65.30x	41.35x	34.13x
MAMBA-Frost-512	67.89x	39.79x	32.38x
MAMBA-Frost-CC-128	38.89x	25.29x	21.17x
MAMBA-Frost-CC-192	47.97x	38.06x	30.70x
MAMBA-Frost-CC-256	56.38x	42.74x	32.65x
MAMBA-Frost-CC-384	48.44x	46.54x	36.13x
MAMBA-Frost-CC-512	41.34x	45.54x	34.46x

7.4 Throughput Results

The following table reports operation throughput in operations per second.

Implementation	Instance	KeyGen	Encaps	Decaps
Reference	MAMBA-Frost-128	100.08	93.90	94.55
Reference	MAMBA-Frost-192	35.18	30.42	34.10
Reference	MAMBA-Frost-256	16.48	16.14	16.03
Reference	MAMBA-Frost-384	7.36	6.77	6.76
Reference	MAMBA-Frost-512	4.07	3.42	3.41
Reference	MAMBA-Frost-CC-128	103.66	94.68	84.45
Reference	MAMBA-Frost-CC-192	35.22	34.66	34.29
Reference	MAMBA-Frost-CC-256	16.63	16.13	16.09
Reference	MAMBA-Frost-CC-384	7.18	7.17	7.19
Reference	MAMBA-Frost-CC-512	3.85	3.88	3.83
Optimized	MAMBA-Frost-128	3707.11	2516.83	1915.79
Optimized	MAMBA-Frost-192	1768.17	1351.76	975.54
Optimized	MAMBA-Frost-256	916.11	672.93	503.19
Optimized	MAMBA-Frost-384	468.08	279.43	227.37
Optimized	MAMBA-Frost-512	267.46	134.46	109.71
Optimized	MAMBA-Frost-CC-128	3754.99	2540.92	1915.78
Optimized	MAMBA-Frost-CC-192	1722.91	1394.46	1043.96
Optimized	MAMBA-Frost-CC-256	935.25	698.90	505.64
Optimized	MAMBA-Frost-CC-384	336.80	326.03	255.71
Optimized	MAMBA-Frost-CC-512	156.39	174.56	131.35

8 Resource Consumption

8.1 Communication and Storage Sizes

Instance	Public key	Secret key	Shared secret	Ciphertext
MAMBA-Frost-128	5152	6736	16	5192
MAMBA-Frost-192	9712	11528	24	9760
MAMBA-Frost-256	16776	19416	32	15552
MAMBA-Frost-384	25096	29032	48	37736
MAMBA-Frost-512	36432	41728	64	72944
MAMBA-Frost-CC-128	5152	6736	16	5192
MAMBA-Frost-CC-192	9712	11528	24	9760
MAMBA-Frost-CC-256	16776	19416	32	15552
MAMBA-Frost-CC-384	37628	43492	48	25204
MAMBA-Frost-CC-512	72832	83328	64	36544

All sizes are identical for the Reference and Optimized implementations because both implementations expose the same API_PKC instance parameters and serialization formats.

8.2 Binary Size, Peak RSS, and Stack Usage

The self-evaluation harness records binary section sizes and peak resident set size. Stack-usage extraction was not enabled in this run and is therefore reported as **N/A**.

Implementation	Instance	Text	Data	BSS	Dec	Peak RSS	Stack
Reference	MAMBA-Frost-128	30,760	752	208	31,720	3,104	N/A
Reference	MAMBA-Frost-192	31,136	752	208	32,096	5,184	N/A
Reference	MAMBA-Frost-256	31,216	752	208	32,176	8,668	N/A
Reference	MAMBA-Frost-384	31,456	752	208	32,416	17,036	N/A
Reference	MAMBA-Frost-512	31,680	752	208	32,640	29,248	N/A
Reference	MAMBA-Frost-CC-128	30,760	752	208	31,720	3,104	N/A
Reference	MAMBA-Frost-CC-192	31,136	752	208	32,096	5,184	N/A
Reference	MAMBA-Frost-CC-256	31,216	752	208	32,176	8,668	N/A
Reference	MAMBA-Frost-CC-384	31,504	752	208	32,464	16,980	N/A
Reference	MAMBA-Frost-CC-512	31,728	752	208	32,688	29,196	N/A
Optimized	MAMBA-Frost-128	31,370	744	200	32,314	2,048	N/A
Optimized	MAMBA-Frost-192	30,986	744	200	31,930	2,176	N/A
Optimized	MAMBA-Frost-256	33,594	744	200	34,538	2,304	N/A
Optimized	MAMBA-Frost-384	34,834	744	200	35,778	2,688	N/A
Optimized	MAMBA-Frost-512	35,842	744	200	36,786	3,072	N/A
Optimized	MAMBA-Frost-CC-128	31,406	744	200	32,350	2,048	N/A
Optimized	MAMBA-Frost-CC-192	31,022	744	200	31,966	2,176	N/A
Optimized	MAMBA-Frost-CC-256	33,630	744	200	34,574	2,304	N/A
Optimized	MAMBA-Frost-CC-384	35,494	744	200	36,438	2,560	N/A
Optimized	MAMBA-Frost-CC-512	36,054	744	200	36,998	2,944	N/A

Binary section sizes are reported in bytes. Peak RSS is reported in KiB. The `resources.csv` file records `stack_usage_not_measured` for all instances.

9 Dependency Description

Compiler	GCC 11.4.0.
Build tool	GNU Make 4.3.
CMake	CMake 3.22.1.
Randomness	API_PKC <code>drng.c/h</code> , used for KAT generation, API testing, and deterministic benchmark initialization.
Hash/XOF	API_PKC <code>auxfunc.c/h</code> , providing <code>sm3hash</code> , <code>pseudohash</code> , and <code>pseudoXOF</code> .
Third-party cryptographic libraries	None. The API_PKC packaging path does not depend on OpenSSL.
System randomness	None. The API_PKC packaging path does not use <code>/dev/urandom</code> , <code>getrandom</code> , or <code>arc4random</code> .
Optimized instruction sets	x86-64, AVX2, AES-NI.

The forbidden-dependency scan command is:

```
grep -R "RAND_bytes\|OpenSSL\|/dev/urandom\|getrandom\|arc4random" -n \
  API_PKC/Implementations/Reference_Implementation/MAMBA-Frost-* \
  API_PKC/Implementations/Optimized_Implementation/MAMBA-Frost-* || true
```

Result: **PASS**. The scan reports OpenSSL only in documentation comments describing disabled configuration options. No runtime use of OpenSSL, system randomness, or forbidden randomness APIs is present in the API_PKC packaging path.

10 Reproducibility

10.1 Self-Evaluation Harness

The self-evaluation harness is located under:

```
x86_Self_Evaluation/
```

The main one-click script is:

```
x86_Self_Evaluation/run_api_pkc_self_eval.sh
```

The benchmark source is:

```
x86_Self_Evaluation/benchmark_kem_api.c
```

The benchmark measures the final API_PKC wrapper functions and does not directly call the internal development API.

10.2 Formal Reproduction Commands

The run reported in this document used:

```
bash x86_Self_Evaluation/run_api_pkc_self_eval.sh --runs 100 --warmup 100 --full-check
```

The result directory is:

```
x86_Self_Evaluation/results/20260628-150128/
```

After extracting the submission package, the build and KAT checks can be reproduced with:

```
cd API_PKC
make clean
make
make smoke
make kats
make kat-repro
make optimized-clean
make optimized-all
make optimized-smoke
make optimized-kats
make optimized-kat-repro
```

The full correctness checks can be reproduced with:

```
cd API_PKC
make check
make optimized-check
```

The reported run enabled these full checks, and both the Reference and Optimized full checks passed.

Build-artifact cleanup check:

```
find API_PKC -type f \( \
  -name "*.o" -o -name "*.a" -o -name "*.so" -o -name "*.dll" -o -name "*.exe" -o \
  -name "*.pdb" -o -name "KAT_KEM" -o -name "test_kem_api" -o -name "perf_kem_api" -o \
  -name "check_avx2" -o -name "core" -o -name ".DS_Store" -o -name "*.pyc" -o -name "
    perf_*.csv" \
\) -print

find API_PKC -type d \( -name "output" -o -name "output.first" -o -name "__pycache__" -
  o -name "build" -o -name "CMakeFiles" \) -print
```

Expected result after cleanup: no output. The reported run produced an empty `artifact_scan_after_clean` the artifact scan therefore passes.

10.3 Raw Logs and Evidence Files

The raw logs of this self-evaluation are stored under:

x86_Self_Evaluation/results/20260628-150128/
--

The main generated evidence files are:

- `environment.txt`
- `git_status.txt`
- `package_layout.txt`
- `status.env`
- `build_reference.log`
- `build_optimized.log`
- `smoke_reference.log`
- `smoke_optimized.log`
- `check_reference.log`
- `check_optimized.log`
- `kat_reference.log`
- `kat_optimized.log`
- `kat_repro_reference.log`
- `kat_repro_optimized.log`
- `benchmark_reference.log`
- `benchmark_optimized.log`
- `benchmark.csv`
- `throughput.csv`
- `sizes.csv`
- `resources.csv`
- `dependency_scan.log`
- `drng_scan.log`
- `auxfunc_scan.log`
- `artifact_scan_before_clean.log`
- `artifact_scan_after_clean.log`
- `summary.md`

The `git_status.txt` file was captured at run time before the final cleanup step and therefore records temporary KAT and benchmark artifacts. The post-cleanup artifact scan is the authoritative artifact-cleanliness check for the reported run.

11 Self-Evaluation Conclusion

Based on the tests described in this report:

- The API_PKC package covers ten MAMBA-Frost KEM instances, consisting of five default instances and five compact-ciphertext variant instances.
- Both the Reference implementation and the performance-optimized implementation can be built independently.
- All ten instances generate KAT files using the API_PKC KAT generator.
- KAT reproducibility passes for both the Reference and Optimized implementations.
- The Reference and Optimized full correctness checks pass for all ten instances.
- The formal KAT files are stored under `API_PKC/Test_Vector/`.
- The API wrapper implements length queries, key generation, encapsulation, and decapsulation.
- `kem_dec` preserves the implicit-rejection behavior required by the KEM design.
- The API_PKC packaging path uses the official DRNG and official auxiliary functions.
- The API_PKC packaging path does not depend on OpenSSL or system randomness.
- The dependency scan reports **PASS**.
- The size check reports **PASS**.
- The build-artifact scan after cleanup reports **PASS**.
- The performance results are reported as median cycle counts over 100 measured runs after 100 warm-up runs.
- Stack usage was not measured in this run.

Final conclusion:

PASS for the executed self-evaluation items. Under the test environment and methodology described in this report, the MAMBA-Frost x86 Reference implementation and x86 AVX2 performance-optimized implementation pass build tests, smoke correctness tests, full correctness checks, KAT generation, KAT reproducibility, API-level benchmark execution, size checks, dependency checks, and artifact-cleanliness checks. The reported package includes both default MAMBA-Frost instances and compact-ciphertext variant instances. Stack-usage extraction is not included in this run and is reported as such.

A Appendix A: Command Summary

A.1 One-Click Self-Evaluation

```
bash x86_Self_Evaluation/run_api_pkc_self_eval.sh --runs 100 --warmup 100 --full-check
```

A.2 Reference Build, Smoke, KAT, and Reproducibility

```
cd API_PKC
make clean
make
make smoke
make kats
make kat-repro
```

A.3 Reference Full Correctness Check

```
cd API_PKC
make check
```

A.4 Optimized Build, Smoke, KAT, and Reproducibility

```
cd API_PKC
make optimized-clean
make optimized-all
make optimized-smoke
make optimized-kats
make optimized-kat-repro
```

A.5 Optimized Full Correctness Check

```
cd API_PKC
make optimized-check
```

A.6 Forbidden Dependency Scan

```
grep -R "RAND_bytes\\|OpenSSL\\|/dev/urandom\\|getrandom\\|arc4random" -n \
  API_PKC/Implementations/Reference_Implementation/MAMBA-Frost-* \
  API_PKC/Implementations/Optimized_Implementation/MAMBA-Frost-* || true
```

A.7 Build-Artifact Scan

```
find API_PKC -type f \( \
  -name "*.o" -o -name "*.a" -o -name "*.so" -o -name "*.dll" -o -name "*.exe" -o \
  -name "*.pdb" -o -name "KAT_KEM" -o -name "test_kem_api" -o -name "perf_kem_api" -o \
  -name "check_avx2" -o -name "core" -o -name ".DS_Store" -o -name "*.pyc" -o -name "
    perf_*.csv" \
  \) -print

find API_PKC -type d \( -name "output" -o -name "output.first" -o -name "__pycache__" -
  o -name "build" -o -name "CMakeFiles" \) -print
```

B Appendix B: Raw Benchmark Data

The raw benchmark data are stored in:

```
x86_Self_Evaluation/results/20260628-150128/benchmark.csv
```

For each implementation, instance, and KEM operation, the CSV file records the timestamp, implementation type, instance name, security level, operation, warm-up runs, measurement runs, mean cycles, median cycles, minimum cycles, maximum cycles, standard deviation, wall-clock timing, throughput, object sizes, status, and notes.

C Appendix C: Raw Environment and Test Logs

The raw environment and test logs are stored in:

```
x86_Self_Evaluation/results/20260628-150128/
```

The run summary reports:

```
Runs: 100
Warmup: 100
AVX2: yes
AES-NI: yes
Reference build: PASS
Reference smoke: PASS
Reference KAT: PASS
Reference kat-repro: PASS
Reference full check: PASS
Optimized build: PASS
Optimized smoke: PASS
Optimized KAT: PASS
Optimized kat-repro: PASS
Optimized full check: PASS
Benchmark: PASS
Dependency scan: PASS
Sizes check: PASS
Artifact scan: PASS
```